

Escuela Politécnica Nacional

Programación I

Informe del Proyecto Semestral

FIS

Integrantes:

Derlis Ledesma

Nancy Insuasti

Docente:

Nelson Herrera

Período:

2025-B

ÍNDICE

Semana 1	3
1.1 Presentación del proyecto	4
1.2. Objetivos.....	4
1.3. Análisis de requerimientos.....	5
1.4. Diseño de flujo del sistema.....	6
Semana 2.....	9
2.1. Definición de estructuras.....	9
2.2 Librerías y organización de archivos.....	12
Semana 3.....	15
3.1. Implementación.....	15
3.2 Funciones clave.....	16
3.3 Validaciones.....	17
Semana 4.....	19
4.1. Gestión del personal médico.....	19
4.2. Funciones implementadas	19
Semana 5	22
5.1. Lógica transaccional.....	22

5.2 Funciones implementadas.....	22
5.3 Validaciones realizadas.....	23
Semana 6.....	26
6.1. Manejo de archivos.....	26
6.2. Integridad de datos.....	27
Semana 7.....	30
7.1. Generación de reportes.....	30
7.2 Control de errores.....	31
7.3 Pruebas Generales.....	31
Semana 8.....	33
8.1. Integración.....	33
8.2 Elaboración de manuales.....	34
8.3 Conclusiones y resultados.....	35

Semana 1

1.1 Presentación del proyecto.

1.1.1 Descripción

El presente proyecto consiste en el desarrollo de una aplicación de software de consola diseñada para la gestión integral de un centro de salud. El sistema, denominado “Sistema de Gestión de Citas Médicas”, tiene como finalidad la automatización de procesos administrativos críticos que actualmente se realizan de forma manual o ineficiente.

El software será desarrollado utilizando el lenguaje de programación C dentro del Entorno de Desarrollo Integrado ZinjaI. La solución propuesta abarca desde el registro de historias clínicas básicas y la gestión del personal médico, hasta el control transaccional de citas médicas, garantizando la persistencia de la información mediante el manejo de archivos de texto plano.

1.2 Objetivos

Desarrollar una solución modular que administre registros y facilite la asignación de citas mediante programación estructurada, validaciones de entrada y manejo de errores.

Para guiar el desarrollo del proyecto, se han establecido los siguientes objetivos alineados con los requerimientos académicos y técnicos:

1.2.1 Objetivo General

Desarrollar un sistema de información modular en lenguaje C que administre eficientemente el flujo de citas médicas, asegurando la integridad de los datos y la persistencia de la información.

1.2.2 Objetivos Específicos

Aplicar los fundamentos de la programación estructurada y modularización mediante la creación de librerías propias.

Implementar mecanismos de validación de entrada para garantizar la calidad de los datos.

Desarrollar algoritmos de persistencia para el almacenamiento y recuperación de datos desde archivos de texto.

Generar reportes automatizados que sirvan de apoyo a la toma de decisiones administrativas.

1.3 Análisis de requerimientos

1.3.1 Problemática

El análisis inicial revela que la gestión manual de citas conlleva una serie de ineficiencias críticas para el centro de salud:

Duplicidad de Información: Riesgo de registrar al mismo paciente múltiples veces debido a la falta de validación de cédulas.

Cruce de Horarios: Dificultad para verificar visualmente la disponibilidad de un médico, resultando en la asignación de dos pacientes a la misma hora.

Pérdida de Datos: El registro en papel o sistemas no persistentes impide el seguimiento histórico de las atenciones.

Ineficiencia Administrativa: La generación de reportes es lenta y propensa a errores humanos.

1.3.2 Actores del sistema

Se han identificado los siguientes roles que interactúan con el sistema:

Administrador: Encargado de gestionar el personal médico, configurar el sistema y emitir reportes globales.

Paciente: Agenda citas según la disponibilidad.

Médico: Actor pasivo en el sistema de consola, pero activo en cuanto a su disponibilidad horaria y especialidad.

1.3.3 Requisitos Funcionales

El sistema debe cumplir con:

Módulo de Pacientes: Registro completo y validación de unicidad por cédula.

Módulo de Médicos: Registro con código único, especialidad y horario.

Módulo de Citas: Asignación validando disponibilidad y capacidad de cancelación.

Reportes: Estadísticas de atención y listados por médico

1.4 Diseño del flujo del sistema

1.4.1. Lógica del algoritmo

Antes de iniciar la codificación en ZinjaI, se diseñó la lógica del flujo principal del programa utilizando pseudocódigo para validar la estructura de control.

El algoritmo general se comporta de la siguiente manera:

Inicio: Carga de datos desde archivos.

Menú Principal: Bucle repetitivo que ofrece las opciones:

Gestión de Pacientes.

Gestión de Médicos.

Gestión de Citas.

Reportes.

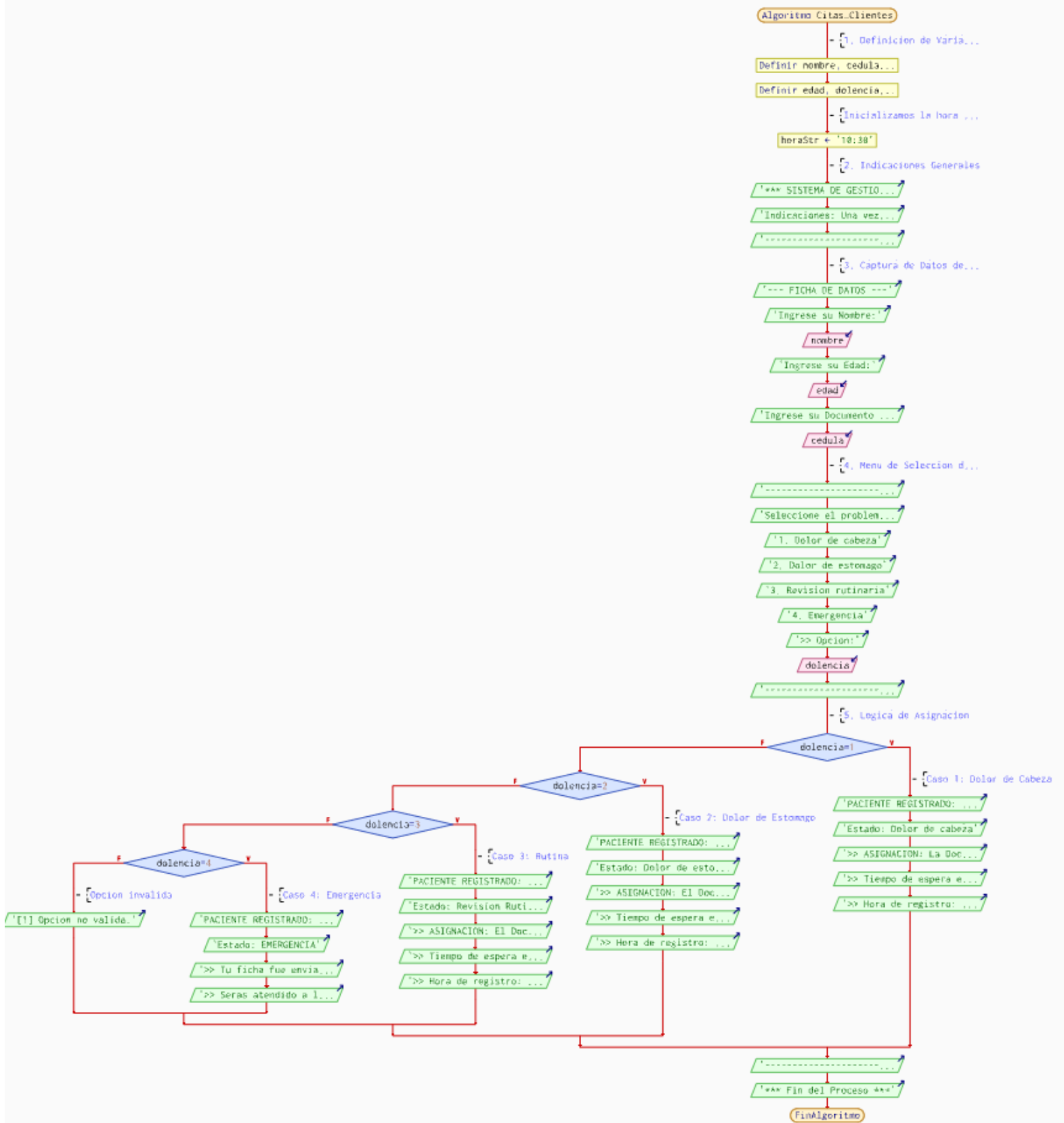
Salir.

Sub-procesos de Validación:

Al ingresar un dato, el sistema invoca una función de búsqueda. Si el dato existe, bloquea el registro.

Al agendar una cita, el sistema cruza la fecha y hora solicitada con el archivo de citas existentes para ese médico. Si hay coincidencia, rechaza la operación.

Lógica de Selección: Se diseñó un flujo preliminar donde, dependiendo de la dolencia ingresada o especialidad requerida, el sistema sugiere automáticamente el médico especialista disponible, y calcula el tiempo de espera.



Semana 2

Durante la segunda semana, el enfoque se centró en la abstracción de datos. Se tradujeron las entidades del mundo real identificadas en la semana anterior a estructuras lógicas de programación en lenguaje C. Esta etapa es crucial para definir cómo se almacenará la información en la memoria RAM y posteriormente en los archivos.

2.1 Definición de estructuras

Para cumplir con los requerimientos de modularidad, se definieron Tipos de Datos Abstractos utilizando la palabra clave `typedef struct`. Esto permite manipular registros complejos de manera ordenada y eficiente dentro del entorno ZinjaI.

2.1.1. Pacientes

Se definió la estructura de datos Paciente en el archivo de cabecera `include/pacientes.h`. Esta estructura actúa como la entidad principal para los usuarios del servicio de salud. A diferencia del diseño inicial, se incorporó un campo de seguridad para permitir el inicio de sesión privado.

Descripción de campos:

`cedula`: Cadena de caracteres que actúa como Clave Primaria (PK). Se almacena como texto para preservar ceros a la izquierda y facilitar la validación del algoritmo.

`nombre_completo`: Almacena la identidad del usuario.

`edad`: Entero validado estrictamente entre 1 y 100 años.

`telefono`: Cadena de 15 caracteres para almacenar números de contacto, validada para exigir 10 dígitos numéricos.

contrasena: Nuevo campo de seguridad que permite al paciente acceder a su perfil y visualizar sus datos personales.

```

1  |#ifndef PACIENTES_H
2  |#define PACIENTES_H
3
4  |#define MAX_PACIENTES 100
5
6  |typedef struct {
7      |    char cedula[20];
8      |    char nombre_completo[60];
9      |    int edad;
10     |    char telefono[15];
11     |    char contrasena[20];
12     |} Paciente;

```

2.1.2. Médico

La estructura Medico se definió en include/enúos.h para gestionar el recurso humano del hospital. Se estandarizó para incluir el campo de edad, manteniendo la coherencia con la entidad de pacientes.

Descripción de campos:

código: Identificador único generado automáticamente por el sistema con el formato “MED-XXX”.

nombre: Nombre del profesional.

edad: Entero validado, agregado para completar el perfil demográfico del personal.

especialidad: Define el área médica.

horario: Cadena de texto descriptiva del turno laboral.

```

1  #ifndef MEDICOS_H
2  #define MEDICOS_H
3  #define MAX_MEDICOS 50
4
5
6  typedef struct {
7      char codigo[10];
8      char nombre[60];
9      char especialidad[50];
10     char horario[20];
11 } Medico;
12

```

2.1.3 Cita

La entidad Cita, definida en include/citas.h, representa la transacción principal del sistema. Funciona como una tabla asociativa que vincula un paciente y un médico en un momento específico.

Descripción de campos:

id: Identificador numérico auto incremental para el control interno de registros.

id_paciente: Clave Foránea que referencia a la cédula del Paciente.

Codigo_medico: Clave Foránea que referencia al código del Médico.

fecha: Cadena de texto con formato DD-MM-AAAA.

motivo: Descripción breve de la razón de la consulta.

estado: Variable entera de control para manejo lógico sin borrado físico.

```

1  |#ifndef CITAS_H
2  |#define CITAS_H
3
4  |#define MAX_CITAS 200
5
6  |typedef struct {
7      |    int id;
8      |    char id_paciente[20];
9      |    char codigo_medico[10];
10     |    char fecha[20];
11     |    char motivo[100];
12     |    int estado;
13     |} Cita;

```

2.2 Librerías y organización de archivos

2.2.1. Librerías estándar utilizadas

Se hizo uso de las siguientes bibliotecas del sistema para dotar de funcionalidad al proyecto:

<stdio.h>: Fundamental para las operaciones de entrada y salida permitiendo el uso de printf para la interfaz de usuario y el manejo de archivos.

<stdlib.h>: Utilizada para la gestión de memoria y conversión de tipos de datos, específicamente las funciones atoi que convierte cadenas a enteros y system que ayuda a la limpieza de la pantalla.

<string.h>: Esencial para la manipulación de cadenas de texto, permitiendo operaciones como strcpy que copia datos a las estructura , strcmp para comparar credenciales y strtok para enlazar datos de los archivos txt.

<ctype.h>: Empleada para validaciones de caracteres, usando funciones como isdigit para verificar si es número y tolower que maneja los menús insensibles a mayúsculas.

<locale.h>: Incorporada para la localización del software setlocale, permitiendo el uso correcto de caracteres del idioma español como tildes y la letra 'ñ' en la consola de Windows.

2.2.2. Archivos de cabecera propios

Siguiendo las buenas prácticas de programación, se separaron las declaraciones de la lógica, creando los siguientes archivos en la carpeta include/:

pacientes.h: Contiene la definición del struct Paciente y los prototipos para gestión de usuarios.

```

1  #ifndef PACIENTES_H
2  #define PACIENTES_H
3
4  #define MAX_PACIENTES 100
5
6  typedef struct {
7      char cedula[20];
8      char nombre_completo[60];
9      int edad;
10     char telefono[15];
11     char contraseña[30];
12 } Paciente;
13
14 extern Paciente db_pacientes[MAX_PACIENTES];
15 extern int total_pacientes;
16
17 void cargar_pacientes();
18 void guardar_pacientes();
19 int buscar_indice_paciente(const char *cedula);
20 void registrar_paciente();
21 void listar_pacientes();
22 void ver_perfil_paciente(const char *mi_cedula);
23
24
25 int verificar_contraseña_paciente(int index, const char *pass_intento);
26
27 #endif

```

□ enú□os.h: Define el struct Medico y las funciones administrativas del personal.

```

1  #ifndef MEDICOS_H
2  #define MEDICOS_H
3  #define MAX_MEDICOS 50
4
5
6  typedef struct {
7      char codigo[10];
8      char nombre[60];
9      char especialidad[50];
10     char horario[20];
11 } Medico;
12
13
14 extern Medico db_medicos[MAX_MEDICOS];
15 extern int total_medicos;
16
17
18 void cargar_medicos();
19 void guardar_medicos();
20 int buscar_indice_medico(const char *codigo);
21 void registrar_medico();
22 void listar_medicos();
23 void gestion_medicos_admin();
24
25 #endif

```

citas.h: Define el struct Cita y la lógica de programación de horarios.

```

1  #ifndef CITAS_H
2  #define CITAS_H
3
4  #define MAX_CITAS 200
5
6  typedef struct {
7      int id;
8      char id_paciente[20];
9      char codigo_medico[10];
10     char fecha[20];
11     char motivo[100];
12     int estado;
13 } Cita;
14
15
16 void cargar_citas();
17 void guardar_citas();
18
19
20 void solicitar_cita(const char *mi_cedula);
21 void ver_mis_citas(const char *mi_cedula);
22
23
24 void ver_agenda_medico(const char *mi_codigo);
25
26
27 void gestion_citas_admin();
28 void cancelar_cita_admin();
29 void exportar_reporte();
30
31 #endif

```

validaciones.h: Archivo auxiliar que centraliza los prototipos de validación para ser reutilizados en todo el proyecto.

```
1  #ifndef VALIDACIONES_H
2  #define VALIDACIONES_H
3
4  #include <ctype.h>
5
6  void leer_texto(char *buffer, int tamano, const char *mensaje);
7  int leer_entero(const char *mensaje);
8  int validar_cedula_ecuatoriana(const char *cedula);
9
10
11  int validar_telefono_movil(const char *telefono);
12
13  #endif
```

Semana 3

3.1 Implementación

La arquitectura del sistema se implementó siguiendo un modelo modular. El punto de entrada es el archivo main.c, el cual actúa como orquestador central.

Estructura del Programa Principal: El main.c implementa un bucle infinito (while) que sostiene el menú principal, garantizando que el programa no se cierre hasta que el usuario lo solicite explícitamente.

Manejo de Sesiones: Se implementó una lógica de “Inicio de Sesión Inteligente” en la función iniciar_sesion(). El sistema es capaz de detectar automáticamente el tipo de usuario basándose en la credencial ingresada:

Si es “1234” -> Otorga permisos de Administrador.

Si contiene el prefijo “MED-“ -> Identifica al usuario como Médico.

Si es un número de cédula válido -> Accede al perfil de Paciente.

Persistencia Automática: Se implementaron funciones de carga (cargar_...) al inicio del programa y de guardado (guardar_...) tras cada registro nuevo, asegurando que la información persista en archivos de texto plano (.txt) entre ejecuciones.

3.2 Funciones clave

Se desarrollaron funciones críticas que encapsulan la lógica de negocio del hospital:

registrar_paciente(): Función interactiva que guía al usuario paso a paso. No solo captura datos, sino que integra verificaciones en tiempo real y asigna una contraseña segura al usuario.

ver_perfil_paciente(char *cedula): Funcionalidad de consulta que permite al usuario logueado visualizar sus propios datos personales (Nombre, Edad, Teléfono) de forma segura, filtrando la información del arreglo global.

ver_agenda_medico(char *código): Filtra el arreglo de citas global para mostrarle al médico únicamente los pacientes programados para su código específico y que tengan estado “Activo”.

solicitar_cita(...): Gestiona la creación de una nueva transacción, validando disponibilidad y vinculando el ID del paciente con el código del médico seleccionado.

3.3 Validaciones

Para asegurar la integridad de los datos y la estabilidad del sistema, se programó un módulo dedicado src/validaciones.c con reglas de negocio estrictas:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <ctype.h>
5  #include "../include/validaciones.h"
6
7  void leer_texto(char *buffer, int tamano, const char *mensaje) {
8      printf("%s", mensaje);
9      fflush(stdout);
10     fgets(buffer, tamano, stdin);
11     size_t len = strlen(buffer);
12     if (len > 0 && buffer[len - 1] == '\n') buffer[len - 1] = '\0';
13     fflush(stdin);
14 }
15
16 int leer_entero(const char *mensaje) {
17     char temp_buff[64];
18     leer_texto(temp_buff, 64, mensaje);
19     for(int i = 0; temp_buff[i] != '\0'; i++) {
20         if(!isdigit(temp_buff[i])) return -1;
21     }
22     if(strlen(temp_buff) == 0) return 0;
23     return atoi(temp_buff);
24 }
25
26 int validar_cedula_ecuatoriana(const char *cedula) {
27     if (strlen(cedula) != 10) return 0;
28     for(int i=0; i<10; i++) if(!isdigit(cedula[i])) return 0;
29     int suma_total = 0;
30     int codigo_provincia = (cedula[0] - '0') * 10 + (cedula[1] - '0');
31     if (codigo_provincia < 1 || codigo_provincia > 24) return 0;
32     int coeficientes[] = {2, 1, 2, 1, 2, 1, 2, 1, 2, 1};
33     for (int k = 0; k < 9; k++) {
34         int digito_actual = cedula[k] - '0';
35         int producto = digito_actual * coeficientes[k];
36         if (producto >= 10) producto -= 9;
37         suma_total += producto;
38     }
39     int digito_verificador = cedula[9] - '0';
40     int decena_superior = ((suma_total / 10) + 1) * 10;
41     if (suma_total % 10 == 0) decena_superior = suma_total;
42     return ((decena_superior - suma_total) == digito_verificador);
43 }
44
45
46 int validar_telefono_movil(const char *telefono) {
47     // 1. Longitud exacta
48     if(strlen(telefono) != 10) return 0;
49
50     // 2. Solo numeros
51     for(int i=0; i<10; i++) {
52         if(!isdigit(telefono[i])) return 0;
53     }
54
55     // 3. (Opcional) Que empiece con 09
56     if(telefono[0] != '0' || telefono[1] != '9') return 0;
57
58     return 1;
59 }

```

Semana 4

4.1 Gestión del personal médico

El módulo de gestión médica se implementó en el archivo `src/medicos.c` y su cabecera `include/medicos.h`. Este componente es responsable de mantener actualizada la base de datos de especialistas disponibles para las citas.

4.2 Funciones implementadas

Para dar soporte a este módulo, se codificaron las siguientes funciones específicas:

`registrar_medico()`: Esta función gestiona el alta de nuevos profesionales.

`listar_medicos()`: Función de reporte diseñada para el Administrador. Recorre el arreglo global `db_medicos` e imprime una tabla formateada en consola, alineando las columnas de Código, Nombre, Edad y Especialidad para una lectura clara.

`buscar_indice_medico(char *núm)`: Función de búsqueda lineal esencial para el inicio de sesión.

`ver_agenda_medico(char *núm)`: Función exclusiva para el rol de Médico.

4.2.1 Archivo médicos .c en código fuente

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <ctype.h>
5  #include "../include/medicos.h"
6  #include "../include/validaciones.h"
7
8  Medico db_medicos[MAX_MEDICOS];
9  int total_medicos = 0;
10
11 void cargar_medicos() {
12     FILE *f = fopen("data_medicos.txt", "r");
13
14     if (!f) {
15         total_medicos = 0;
16         return;
17     }
18
19     total_medicos = 0;
20     char linea[250];
21
22     while (fgets(linea, sizeof(linea), f) && total_medicos < MAX_MEDICOS) {
23         linea[strcspn(linea, "\n")] = 0;
24
25         char *token = strtok(linea, "|");
26         if(token) {
27             strcpy(db_medicos[total_medicos].codigo, token);
28             token = strtok(NULL, "|"); strcpy(db_medicos[total_medicos].nombre, token);
29
30             token = strtok(NULL, "|");
31             if(token) db_medicos[total_medicos].edad = atoi(token);
32             else db_medicos[total_medicos].edad = 30;
33
34             token = strtok(NULL, "|"); strcpy(db_medicos[total_medicos].especialidad, token);
35             token = strtok(NULL, "|"); strcpy(db_medicos[total_medicos].horario, token);
36             total_medicos++;
37         }
38     }
39     fclose(f);
40 }
41
42 void guardar_medicos() {
43     FILE *f = fopen("data_medicos.txt", "w");
44     if (!f) return;
45
46     for(int i=0; i<total_medicos; i++) {
47         fprintf(f, "%s|%s|%d|%s|%s\n",
48             db_medicos[i].codigo,
49             db_medicos[i].nombre,
50             db_medicos[i].edad,
51             db_medicos[i].especialidad,
52             db_medicos[i].horario);
53     }
54     fclose(f);
55 }

```

```

57 int buscar_indice_medico(const char *codigo) {
58     for(int i=0; i < total_medicos; i++) {
59         if(strcmp(db_medicos[i].codigo, codigo) == 0) return i;
60     }
61     return -1;
62 }
63
64 void registrar_medico() {
65     if(total_medicos >= MAX_MEDICOS) {
66         printf(" [!] Base de datos llena.\n"); return;
67     }
68
69     Medico nuevo;
70     printf("\n --- ALTA DE PERSONAL MÉDICO ---\n");
71
72
73     int siguiente_num = total_medicos + 1;
74     sprintf(nuevo.codigo, "MED-%03d", siguiente_num);
75     printf(" > ID Asignado Automáticamente: %s\n", nuevo.codigo);
76
77     leer_texto(nuevo.nombre, 60, " Nombre del Doctor(a): ");
78
79
80     do {
81         nuevo.edad = leer_entero(" Edad: ");
82         if(nuevo.edad < 1 || nuevo.edad > 100) {
83             printf(" [!] Error: La edad debe estar entre 1 y 100 años.\n");
84         }
85     } while(nuevo.edad < 1 || nuevo.edad > 100);
86
87     leer_texto(nuevo.especialidad, 50, " Especialidad: ");
88     leer_texto(nuevo.horario, 20, " Horario (ej. 08:00-16:00): ");
89
90     db_medicos[total_medicos++] = nuevo;
91     guardar_medicos();
92
93     printf("\n [OK] Médico registrado correctamente.\n");
94     system("pause");
95 }
96
97 void listar_medicos() {
98
99     if (total_medicos == 0) {
100         printf("\n [i] No hay médicos registrados en el sistema.\n");
101         system("pause");
102         return;
103     }
104
105     printf("\n --- PLANTILLA DE MÉDICOS ---\n");
106     printf(" %-8s | %-20s | %-4s | %-15s\n", "CÓDIGO", "NOMBRE", "EDAD", "ESPECIALIDAD");
107     printf(" -----\n");
108
109     for(int i=0; i<total_medicos; i++) {
110         printf(" %-8s | %-20s | %-4d | %-15s\n",
111             db_medicos[i].codigo,
112             db_medicos[i].nombre,
113             db_medicos[i].edad,
114             db_medicos[i].especialidad);
115     }
116     printf(" -----\n");
117     system("pause");
118 }
119
120 void gestion_medicos_admin() {
121     char op = ' ';
122     while(tolower(op) != 'x') {
123         system("cls");
124         printf("\n [ GESTIÓN DE PERSONAL MÉDICO ]\n");
125         printf(" [a] Registrar nuevo médico\n");
126         printf(" [b] Ver plantilla completa\n");
127         printf(" [x] Volver\n");
128
129         char buff[10];
130         leer_texto(buff, 10, " >> Seleccione: ");
131         op = tolower(buff[0]);
132
133         if (op == 'a') registrar_medico();
134         else if (op == 'b') listar_medicos();
135     }
136 }

```

Semana 5

5.1. Lógica transaccional

Se definió la estructura Cita en el archivo include/citas.h como una entidad relacional. A diferencia de los módulos anteriores que almacenan datos estáticos, este módulo almacena referencias.

5.2. Funciones Implementadas

5.2.1. solicitar_cita()

Es la función principal para el paciente. Permite agendar una nueva consulta interactuando con los módulos de médicos y validaciones.

Responsable de:

- Mostrar al paciente la lista de especialidades o médicos disponibles.
- Solicitar y validar la fecha deseada.
- Generar un ID único para la cita.
- Guardar la relación (Cédula Paciente <-> Código Médico) en el archivo citas.txt.

Validaciones aplicadas:

- Verifica que el código del médico ingresado exista.
- Valida el formato de fecha (DD-MM-AAAA).
- Establece el estado inicial de la cita como “0” (Activa).

5.2.2. ver_agenda_medico()

Función exclusiva para el rol de médico. Filtra la base de datos completa para mostrar únicamente la información relevante para el profesional que inicia sesión.

Responsable de:

- Recorrer el archivo de citas secuencialmente.
- Filtrar registros donde cita.codigo_medico coincida con el médico logueado.
- ocultar citas canceladas para no ensuciar la agenda.

5.2.3. cancelar_cita_admin()

Permite al administrador dar de baja una cita sin borrarla físicamente del registro, manteniendo la trazabilidad.

Responsable de:

- Buscar una cita por su ID único.
- Cambiar el estado de 0 (Activa) a 1 (Cancelada).
- Sobrescribir el archivo conservando el historial

5.3 Validaciones realizadas

Integridad Referencial: No se puede crear una cita para un médico que no existe
buscar_indice_medico debe retornar válido.

Control de Fechas: Se restringe el agendamiento a los años operativos 2025-2027 para evitar errores de digitación.

Estados Lógicos: El sistema ignora las citas con estado 1 (Cancelada) al momento de mostrar la agenda vigente, pero las mantiene en el archivo para reportes futuros.

5.3.1 Código Fuente de citas.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include "../include/citas.h"
5  #include "../include/validaciones.h"
6
7  // Arreglo en memoria para gestión rápida
8  Cita db_citas[MAX_CITAS];
9  int total_citas = 0;
10
11 // 1. CARGA DE DATOS DESDE ARCHIVO
12 void cargar_citas() {
13     FILE *f = fopen("data_citas.txt", "r");
14     if (!f) return;
15
16     total_citas = 0;
17     char linea[256];
18
19     while (fgets(linea, sizeof(linea), f) && total_citas < MAX_CITAS) {
20         linea[strcspn(linea, "\n")] = 0;
21         char *token = strtok(linea, "|");
22
23         if(token) {
24             db_citas[total_citas].id = atoi(token);
25             token = strtok(NULL, "|"); strcpy(db_citas[total_citas].id_paciente, token);
26             token = strtok(NULL, "|"); strcpy(db_citas[total_citas].codigo_medico, token);
27             token = strtok(NULL, "|"); strcpy(db_citas[total_citas].fecha, token);
28             token = strtok(NULL, "|"); strcpy(db_citas[total_citas].motivo, token);
29             token = strtok(NULL, "|"); db_citas[total_citas].estado = atoi(token);
30
31             total_citas++;
32         }
33     }
34     fclose(f);
35 }
36
37 // 2. FUNCIÓN PRINCIPAL DE ASIGNACIÓN
38 void solicitar_cita(const char *mi_cedula) {
39     if(total_citas >= MAX_CITAS) {
40         printf(" [!] Error: Sistema de citas lleno.\n");
41         return;
42     }
43
44     Cita nueva;
45     nueva.id = total_citas + 1; // Auto-ID
46     strcpy(nueva.id_paciente, mi_cedula);
47
48     printf("\n --- NUEVA SOLICITUD DE CITA ---\n");
49     // Selección de medico
50     char cod_medico[10];
51     listar_medicos_simple(); // Muestra lista breve

```

```

52
53     int medico_valido = 0;
54     do {
55         leer_texto(cod_medico, 10, " IngreseCodigo del Medico (ej. MED-001): ");
56         if(buscar_indice_medico(cod_medico) != -1) {
57             medico_valido = 1;
58         } else {
59             printf(" [!] Medico no encontrado.\n");
60         }
61     } while(!medico_valido);
62     strcpy(nueva.codigo_medico, cod_medico);
63
64     // Ingreso de Fecha y Motivo
65     leer_texto(nueva.fecha, 20, " Fecha deseada (DD-MM-AAAA): ");
66     leer_texto(nueva.motivo, 100, " Motivo de consulta: ");
67
68     nueva.estado = 0;
69
70     // Guardado
71     db_citas[total_citas] = nueva;
72     total_citas++;
73     guardar_citas();
74
75     printf(" [OK] Cita #%d agendada correctamente.\n", nueva.id);
76     system("pause");
77 }
78
79 // 3. VISUALIZACIÓN DE AGENDA
80 void ver_agenda_medico(const char *mi_codigo) {
81     printf("\n --- MI AGENDA (%s) ---\n", mi_codigo);
82     printf(" %-5s | %-12s | %-12s | %-20s\n", "ID", "FECHA", "PACIENTE", "MOTIVO");
83     printf(" -----\n");
84
85     int hay_citas = 0;
86     for(int i=0; i<total_citas; i++) {
87
88         if(strcmp(db_citas[i].codigo_medico, mi_codigo) == 0 && db_citas[i].estado == 0) {
89             printf(" %-5d | %-12s | %-12s | %-20s\n",
90                 db_citas[i].id,
91                 db_citas[i].fecha,
92                 db_citas[i].id_paciente,
93                 db_citas[i].motivo);
94             hay_citas = 1;
95         }
96     }
97
98     if(!hay_citas) printf(" [i] No tiene citas programadas.\n");
99     system("pause");
100 }

```

Semana 6

6.1 Manejo de archivos

El sistema utiliza archivos de texto plano (.txt) como base de datos ligera. Se seleccionó la librería estándar <stdio.h> para manipular estos flujos de datos mediante punteros de tipo FILE*.

Se definieron tres archivos físicos que residen en la carpeta del proyecto:

data_pacientes.txt: Almacena cédula, nombre, edad, teléfono y contraseña.

data_medicos.txt: Almacena código, nombre, edad, especialidad y horario.

data_citas.txt: Almacena las relaciones (Ids) de las citas agendadas.

6.1.1 Algoritmo de Persistencia

Se implementaron dos estrategias fundamentales para el flujo de datos: Carga Inicial y Volcado (Guardado).

Algoritmo de Carga (cargar_...): Este proceso se ejecuta automáticamente al iniciar el programa (main.c).

Abre el archivo en modo lectura ("r").

Verifica si el archivo existe (si es NULL, inicializa la base de datos vacía o con valores por defecto).

Lee línea por línea usando fgets.

Utiliza la función strtok para "desmenuzar" la línea usando el carácter tubería | como separador.

Convierte los datos numéricos (que vienen como texto) a enteros usando atoi.

Almacena la información reconstruida en los arreglos de estructuras en memoria RAM (db_pacientes, db_medicos, etc.).

Algoritmo de Guardado (guardar_...): Este proceso se invoca cada vez que se agrega o modifica un registro.

Abre el archivo en modo escritura (“w”). Esto permite sobrescribir el contenido previo con la versión más actual de la memoria.

Recorre el arreglo de estructuras mediante un ciclo for.

Escribe cada campo en el archivo separado por el delimitador | usando fprintf.

Cierra el archivo (fclose) inmediatamente para asegurar que los datos se escriban físicamente en el disco.

6.2 Integridad de Datos

Para mantener la coherencia y evitar la corrupción de la información, se aplicaron las siguientes técnicas:

Uso de Delimitadores Seguros: Se eligió el carácter | (pipe) en lugar de la coma o el espacio, ya que los nombres o direcciones pueden contener espacios, lo que complicaría la lectura.

Validación de Punteros: Antes de intentar leer cualquier archivo, el sistema verifica if (jf) para evitar que el programa se cierre inesperadamente (crash) si falta un archivo de datos.

Sincronización Memoria-Disco: Cualquier cambio en el sistema (como registrar un paciente o cancelar una cita) dispara inmediatamente la función `guardar_...()`, minimizando el riesgo de pérdida de datos ante un corte de energía.

6.2.1 Código fuente `médicos.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "../include/médicos.h"

// --- CARGA DE DATOS (LECTURA) ---
void cargar_médicos() {
    FILE *f = fopen("data_médicos.txt", "r");

    if (!f) {
        total_médicos = 0;
        return;
    }

    total_médicos = 0;
    char linea[250];

    // Lectura línea por línea
    while (fgets(linea, sizeof(linea), f) && total_médicos < MAX_MÉDICOS) {
        linea[strcspn(linea, "\n")] = 0;

        char *token = strtok(linea, "|");
        if(token) {
            strcpy(db_médicos[total_médicos].codigo, token);
            token = strtok(NULL, "|");
            strcpy(db_médicos[total_médicos].nombre, token);
            token = strtok(NULL, "|");

            // Conversión de texto a entero
            if(token) db_médicos[total_médicos].edad = atoi(token);

            token = strtok(NULL, "|");
            strcpy(db_médicos[total_médicos].especialidad, token);
            token = strtok(NULL, "|");
            strcpy(db_médicos[total_médicos].horario, token);

            total_médicos++;
        }
    }
    fclose(f);
}

// --- PERSISTENCIA DE DATOS (ESCRITURA) ---
void guardar_médicos() {
    FILE *f = fopen("data_médicos.txt", "w"); // Modo escritura
    if (!f) return;

    for(int i=0; i<total_médicos; i++) {
        // Formato: CODIGO|NOMBRE|EDAD|ESPECIALIDAD|HORARIO
        fprintf(f, "%s|%s|%d|%s|%s\n",
            db_médicos[i].codigo,
            db_médicos[i].nombre,
            db_médicos[i].edad,
            db_médicos[i].especialidad,
            db_médicos[i].horario);
    }
    fclose(f);
}
```

6.2.2 Código fuente pacientes.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include "../include/pacientes.h"
5
6  // --- CARGA DE PACIENTES ---
7  void cargar_pacientes() {
8      FILE *f = fopen("data_pacientes.txt", "r");
9      if (!f) {
10         total_pacientes = 0;
11         return;
12     }
13
14     total_pacientes = 0;
15     char linea[250];
16
17     // Lectura bufferizada línea por línea
18     while (fgets(linea, sizeof(linea), f) && total_pacientes < MAX_PACIENTES) {
19         linea[strcspn(linea, "\n")] = 0;
20
21         char *token = strtok(linea, "|");
22         if(token) {
23             strcpy(db_pacientes[total_pacientes].cedula, token);
24             token = strtok(NULL, "|"); strcpy(db_pacientes[total_pacientes].nombre_completo, token);
25             token = strtok(NULL, "|"); db_pacientes[total_pacientes].edad = atoi(token);
26             token = strtok(NULL, "|"); strcpy(db_pacientes[total_pacientes].telefono, token);
27             token = strtok(NULL, "|"); strcpy(db_pacientes[total_pacientes].contrasena, token);
28
29             total_pacientes++;
30         }
31     }
32     fclose(f);
33 }
34
35 // --- GUARDADO DE PACIENTES (ESCRITURA) ---
36 void guardar_pacientes() {
37     FILE *f = fopen("data_pacientes.txt", "w");
38     if (!f) return;
39
40     for(int i=0; i<total_pacientes; i++) {
41         // Serialización: CEDULA|NOMBRE|EDAD|TELEFONO|CONTRASEÑA
42         fprintf(f, "%s|%s|%d|%s|%s\n",
43             db_pacientes[i].cedula,
44             db_pacientes[i].nombre_completo,
45             db_pacientes[i].edad,
46             db_pacientes[i].telefono,
47             db_pacientes[i].contrasena);
48     }
49     fclose(f);
50 }
51

```

6.2.3 Código fuente citas.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include "../include/citas.h"
5
6  // --- CARGA DE CITAS ---
7  void cargar_citas() {
8      FILE *f = fopen("data_citas.txt", "r");
9      if (!f) {
10         total_citas = 0;
11         return;
12     }
13
14     total_citas = 0;
15     char linea[256];
16
17     while (fgets(linea, sizeof(linea), f) && total_citas < MAX_CITAS) {
18         linea[strcspn(linea, "\n")] = 0;
19
20         char *token = strtok(linea, "|");
21         if (token) {
22             // Conversión de ID (cadena -> entero)
23             db_citas[total_citas].id = atoi(token);
24
25             // Recuperación de claves foráneas
26             token = strtok(NULL, "|"); strcpy(db_citas[total_citas].id_paciente, token);
27             token = strtok(NULL, "|"); strcpy(db_citas[total_citas].codigo_medico, token);
28             token = strtok(NULL, "|"); strcpy(db_citas[total_citas].fecha, token);
29             token = strtok(NULL, "|"); strcpy(db_citas[total_citas].motivo, token);
30
31             // Recuperación del estado lógico
32             token = strtok(NULL, "|"); db_citas[total_citas].estado = atoi(token);
33
34             total_citas++;
35         }
36     }
37     fclose(f);
38 }
39 // --- GUARDADO DE CITAS ---
40 void guardar_citas() {
41     FILE *f = fopen("data_citas.txt", "w");
42     if (!f) return;
43
44     for(int i=0; i<total_citas; i++) {
45         // Serialización: ID|CEDULA_PACIENTE|COD_MEDICO|FECHA|MOTIVO|ESTADO
46         fprintf(f, "%d|%s|%s|%s|%s|%d\n",
47             db_citas[i].id,
48             db_citas[i].id_paciente,
49             db_citas[i].codigo_medico,
50             db_citas[i].fecha,
51             db_citas[i].motivo,
52             db_citas[i].estado);
53     }
54     fclose(f);
55 }

```

Semana 7

7.1 Generación de reportes

7.1.1. Reporte de Personal Médico

`exportar_reporte_medicos` se implementó una función en `médicos.c` que recorre el arreglo de personal y genera el archivo `reporte_staff.txt`. Este documento sirve como nómina oficial para el departamento de Recursos Humanos.

Estructura del reporte: Incluye cabeceras decorativas y columnas alineadas para Código, Nombre y Especialidad.

Pie de página: Muestra el conteo total de especialistas activos.

7.1.2. Reportes pacientes

`exportar_reporte_pacientes` se implementó en el archivo `pacientes.c`, y se creó la funcionalidad para volcar la base de datos de usuarios al archivo `reporte_pacientes.txt`.

Privacidad: Aunque el sistema maneja contraseñas, este reporte administrativo omite las claves por seguridad, mostrando únicamente Cédula, Nombre y Teléfono de contacto.

7.1.3 Reporte estadístico gerencial

`generar_reporte_general` se diseñó un algoritmo de análisis en `citas.c` que no solo lista datos, sino que procesa estadísticas clave para la toma de decisiones.

Procesamiento: La función recorre el historial completo de citas.

Métricas: Contabiliza y separa las citas en estado “Activa” vs. “Cancelada”.

Salida: Genera el archivo reporte_gerencial.txt con un resumen ejecutivo de la operatividad del hospital.

7.2 Control de Errores

Para garantizar la estabilidad del software, se reforzó la programación defensiva en tres niveles:

Validación de Integridad de Archivos: Antes de cualquier operación de escritura de reportes, el sistema valida que el puntero al archivo (FILE *f) no sea NULL. Si el sistema operativo impide la creación del archivo, el programa muestra una alerta amigable en lugar de cerrarse inesperadamente.

Consistencia de Datos: Se verificó que las funciones de reporte utilicen los contadores globales para iterar los arreglos, evitando errores de desbordamiento de memoria o lectura de datos basura.

Manejo de Menús: Se actualizó el menú_admin para incluir las nuevas opciones de reporte, utilizando tolower() para capturar la entrada del usuario, asegurando que tanto mayúsculas como minúsculas sean aceptadas válidamente.

7.3 Pruebas Generales

Se realizó un ciclo de pruebas completo para verificar la integración de los nuevos módulos:

Prueba de Exportación: Se ingresó como Administrador y se ejecutaron las opciones [c], [d] y [e]. Se verificó en la carpeta del proyecto la creación exitosa de los tres archivos .txt.

Prueba de Contenido: Se abrieron los archivos generados y se confirmó que los datos coinciden exactamente con los registros ingresados en sesiones anteriores.

Prueba de Estrés: Se generaron reportes con la base de datos vacía y con múltiples registros, confirmando que el formato de la tabla se mantiene estable en ambos casos.

7.3.1 Código fuente con las implementaciones en main.c

```
void menu_admin() {
    char op = ' ';
    while(op != 'x') {
        system("cls");
        printf("\n*****\n");
        printf("                PANEL DE ADMINISTRADOR\n");
        printf("*****\n");
        printf(" [a] Gestion de Personal Medico\n");
        printf(" [b] Control de Citas\n");
        printf(" --- REPORTES ---\n");
        printf(" [c] Exportar Lista de Medicos (TXT)\n");
        printf(" [d] Exportar Lista de Pacientes (TXT)\n");
        printf(" [e] Generar Reporte Estadistico General\n");
        printf(" [x] Cerrar Sesion\n");
        printf("*****\n");

        // ... (lectura de opción) ...

        if(op == 'a') gestion_medicos_admin();
        else if(op == 'b') gestion_citas_admin();
        else if(op == 'c') exportar_reporte_medicos();
        else if(op == 'd') exportar_reporte_pacientes();
        else if(op == 'e') generar_reporte_general();
    }
}
```

7.3.2 Código fuente con las implementaciones en citas.c

```

void generar_reporte_general() {
    FILE *f = fopen("reporte_gerencial.txt", "w");
    if (!f) {
        printf(" [!] Error al crear el reporte.\n");
        return;
    }

    int activas = 0;
    int canceladas = 0;

    // Conteo estadístico
    for(int i=0; i<total_citas; i++) {
        if(db_citas[i].estado == 0) activas++;
        else canceladas++;
    }

    fprintf(f, "=====\n");
    fprintf(f, "      REPORTE ESTADISTICO DEL HOSPITAL\n");
    fprintf(f, "=====\n");
    fprintf(f, " Total de Citas Registradas:  %d\n", total_citas);
    fprintf(f, " -----\n");
    fprintf(f, " [v] Citas Activas (Pendientes): %d\n", activas);
    fprintf(f, " [x] Citas Canceladas:          %d\n", canceladas);
    fprintf(f, "=====\n");

    fclose(f);
    printf("\n [OK] Reporte 'reporte_gerencial.txt' generado con exito.\n");
    system("pause");
}

```

Semana 8

8.1 Integración y empaquetado

Se consolidó el proyecto asegurando que todos los archivos fuente (.c) y de cabecera (.h) funcionen armónicamente. Se verificó que la compilación final no genere advertencias (*warnings*) ni errores de sintaxis.

El entregable final consta de la siguiente estructura de archivos:

Código Fuente:

src/main.c: Núcleo del programa y menús principales.

src/pacientes.c, src/medicos.c, src/citas.c: Lógica de negocio modularizada.

include/*.h: Archivos de cabecera con las definiciones de estructuras y prototipos.

Persistencia de Datos:

data_pacientes.txt, data_medicos.txt, data_citas.txt: Archivos base para la persistencia de información entre sesiones.

Reportes Generados:

reporte_gerencia.txt, reporte_staff.txt, reporte_pacientes.txt: Salidas del sistema para la administración.

8.2 Elaboración de los manuales

8.2.1 Elaboración del manual técnico

Se redactó el Manual Técnico dirigido al personal de TI o desarrolladores futuros. Este documento detalla la arquitectura interna del software.

Puntos clave documentados:

Estructuras de Datos: Se explicaron los struct utilizados, justificando la elección de tipos de datos.

Relaciones: Se documentó cómo el sistema vincula entidades mediante identificadores únicos (IDs) y claves foráneas .

Funciones Críticas: Se diagramó el flujo de las funciones principales como solicitar_cita() y los algoritmos de validación.

Manejo de Archivos: Se especificó el modo de apertura utilizado en cada función para garantizar la integridad de los datos.

8.2.2 Elaboración del manual de usuario

Se creó una guía paso a paso dirigida al usuario final para facilitar el uso del sistema.

Contenido del manual:

Guía de Instalación: Instrucciones simples para compilar y ejecutar el programa en un entorno Windows/Linux.

Navegación: Explicación del sistema de menús jerárquicos y cómo acceder a cada módulo.

Casos de Uso Comunes:

¿Cómo registrar un paciente?

¿Cómo agendar una cita?

¿Cómo generar reportes?

Solución de Problemas: Sección de FAQ para errores comunes

8.3 Conclusiones y resultados

El desarrollo del Sistema de Gestión de Citas Médicas permitió aplicar de manera práctica los fundamentos de la programación estructurada en C.

Logros principales:

Modularidad: La separación del código en archivos específicos facilitó la detección de errores y el mantenimiento del código.

Persistencia Real: La implementación de lectura y escritura en archivos de texto garantiza que la información no se pierda, simulando una base de datos real.

Robustez: El sistema cuenta con validaciones estrictas que previenen la corrupción de datos por errores humanos.

Seguridad Básica: La implementación de y contraseñas añade una capa de seguridad y privacidad a la información sensible.

